DOCUMENTO DE COBERTURA DE TESTING

Audi F1 Store — Jasmine & Karma Test Suite (EP2) · Versión 2.0

DSY1104 Desarrollo Fullstack II · Evaluación Parcial 2 · 17 de octubre de 2026

1. Introducción y estrategia de pruebas

Este documento detalla la estrategia, la configuración, los casos de prueba y los resultados de cobertura de las pruebas unitarias de la Audi F1 Store. El objetivo es asegurar la lógica de negocio, la manipulación del DOM, la gestión de props y estado, y el comportamiento de los componentes React usando **Jasmine** y **Karma** sobre un navegador real (Chrome Headless).

La estrategia combina tres niveles:

- **Funciones puras** (validadores y fuente de datos): se prueban directamente con entradas y salidas, incluyendo valores límite.
- **Hooks** (`useCarrito`): se prueban aislados con `renderHook` , sin interfaz.
- **Componentes y vistas**: se renderizan con React Testing Library y se simulan las acciones del usuario (escribir, hacer clic, navegar), verificando lo que aparece en el DOM.

2. Configuración del entorno de pruebas

Herramienta	Rol en el proceso
Karma 6 (`karma.config.cjs`)	Test runner: levanta Chrome Headless, carga las pruebas y reporta resultados.
Jasmine 4.6 (cargado por `karma-jasmine` 5.1)	Framework de pruebas: `describe` , `it` , `expect` , `beforeEach` , spies y `jasmine.clock()` .
karma-esbuild	Transpila JSX y módulos ES al vuelo, sin necesidad de Webpack.
React Testing Library	`render` , `renderHook` , `screen` , `fireEvent` , `act` y `cleanup` .
esbuild-plugin-istanbul + `.nycrc.json`	Instrumenta el código de `src/` (excepto `src/tests/`) para contar qué se ejecuta.
karma-coverage	Genera el reporte HTML (`coverage/index.html`) y el resumen en consola. Define umbrales mínimos.

Fragmento clave de la configuración (umbral de cobertura: si no se cumple, `npm test` falla):

```
reporters: ["progress", "coverage"],
coverageReporter: {
  dir: "coverage",
  reporters: [{ type: "html", subdir: "." }, { type: "text-summary" }, { type: "text" }],
  check: { global: { statements: 90, branches: 90, functions: 90, lines: 90 } },
},
```

Además, Karma sirve las imágenes de `public/img` mediante un `proxies`, para que los componentes con `<img>` no generen errores 404 durante las pruebas. Los datos de prueba compartidos y el montaje con router y carrito están en `src/tests/helpers.js`.

3. Resumen del set de pruebas (94 pruebas exitosas)

Archivo de prueba	Módulo evaluado	Casos (it)	Resultado
validadores.spec.js	Validadores puros: correo, RUT, contraseña, login, registro, contacto y checkout	19	**19/19**
productos.spec.js	Fuente de datos simulada: CRUD, casos límite, ofertas y formato de precio	9	**9/9**
CarritoContext.spec.js	Hook `useCarrito`: agregar, eliminar, cupones, totales, persistencia y toast	9	**9/9**
ProductCard.spec.js	Componente: props, renderizado condicional y evento clic	3	**3/3**
Login.spec.js	Componente: estado controlado y mensajes de error	3	**3/3**
Formularios.spec.js	Login por rol, Registro (región/comuna) y Contacto	8	**8/8**
Carrito.spec.js	Vista carrito: vacío, cupones, eliminar y navegación al checkout	6	**6/6**
Checkout.spec.js	Flujo de pago: validación, cupón, pago aprobado y pago rechazado	6	**6/6**
Navegacion.spec.js	Rutas, Navbar responsivo, toast, categorías, 404 y ErrorBoundary	17	**17/17**
Admin.spec.js	Dashboard, stock crítico, CRUD de productos y usuarios, menú móvil	14	**14/14**
Total		**94**	**94/94**

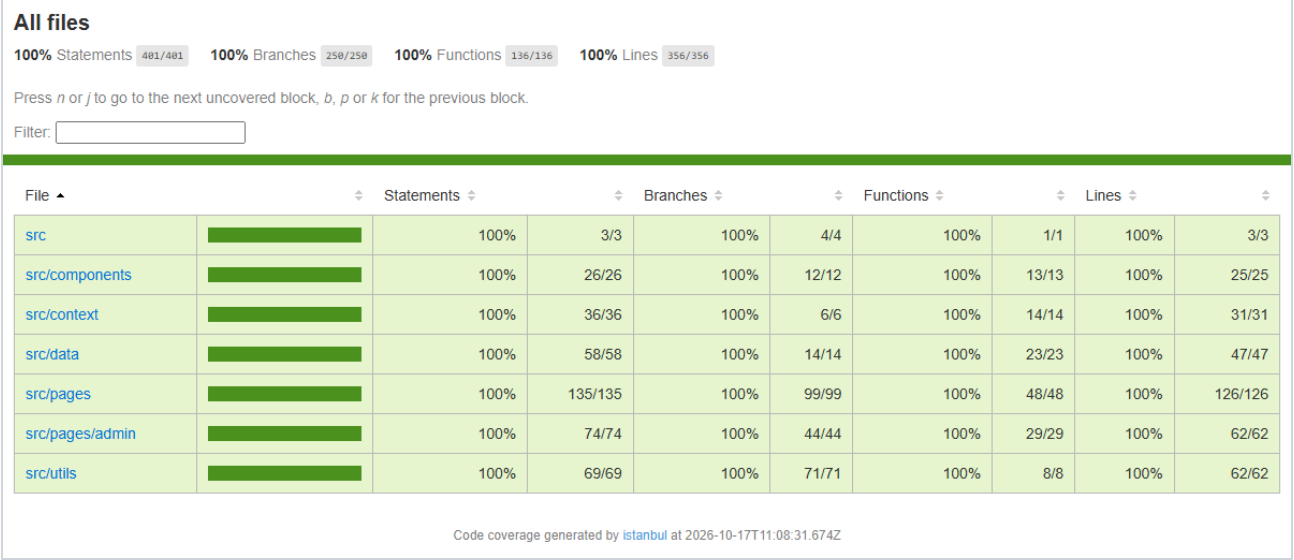
4. Uso de mocks y técnicas de aislamiento

Técnica	Dónde se usa	Para qué
`jasmine.createSpy()`	ProductCard.spec.js	Reemplaza la función `onAgregar` recibida por props y verifica que se llame una vez con el producto y el precio correctos.
`spyOn(Storage.prototype, "getItem").and.returnValue(...)`	CarritoContext.spec.js	Simula un carrito guardado sin depender del almacenamiento real.
`spyOn(..., "setItem" / "removeItem").and.callThrough()`	CarritoContext.spec.js	Espía la persistencia: comprueba qué se guarda y qué se borra, dejando que la llamada ocurra.
`spyOn(window, "confirm").and.returnValue(true/false)`	Admin.spec.js	Simula que el administrador acepta o cancela una eliminación, sin abrir el diálogo real.
`spyOn(window, "scrollTo")` y `spyOn(console, "error")`	Admin.spec.js, Navegacion.spec.js	Anulan efectos secundarios (scroll y mensajes de error esperados) durante la prueba.
`jasmine.clock().install()` + `tick()`	CarritoContext.spec.js	Controla el reloj para probar que el toast se oculta a los 2,5 s sin esperar tiempo real.
`MemoryRouter` con rutas de prueba	Vistas con navegación	Reemplaza el navegador real: permite fijar la URL inicial y verificar a qué pantalla se navega.
`localStorage.clear()` en `beforeEach`	Todas las pruebas con datos	Cada prueba parte del mismo estado: las pruebas son independientes entre sí.

5. Resultados de cobertura

100 %	100 %	100 %	100 %
Sentencias (401/401)	Ramas (250/250)	Funciones (136/136)	Líneas (356/356)

Carpeta	Sentencias	Ramas	Funciones	Líneas
src (App.jsx)	100 %	100 %	100 %	100 %
src/components	100 %	100 %	100 %	100 %
src/context	100 %	100 %	100 %	100 %
src/data	100 %	100 %	100 %	100 %
src/pages	100 %	100 %	100 %	100 %
src/pages/admin	100 %	100 %	100 %	100 %
src/utils	100 %	100 %	100 %	100 %



Reporte HTML generado por karma-coverage (Istanbul) en `coverage/index.html`.

6. Análisis de resultados

- **Todas las pruebas pasan** (94/94) en Chrome Headless, en menos de un segundo de ejecución.
- **Cobertura total (100 %)** en sentencias, ramas, funciones y líneas del código de la aplicación.
- **Proceso iterativo:** la primera medición (22 pruebas) cubría solo los validadores, la fuente de datos, `ProductCard` y `Login`. El reporte mostró las partes sin probar (carrito, checkout, panel de administración y navegación); se agregaron pruebas hasta cubrirlas. Luego el detalle por línea del reporte (*Uncovered Line #s*) mostró 12 ramas defensivas pendientes, como una lista vacía, un id inexistente o un RUT con dígito verificador 0, y también se probaron.

- **Excluidos de la medición:** `src/tests/` (las pruebas mismas) y `src/main.jsx` (solo monta la app en el HTML).
- **Límites:** una cobertura de 100 % indica que todo el código se ejecuta en alguna prueba, no que no existan errores. Por eso las pruebas verifican resultados concretos (textos, montos, navegación) y el diseño responsivo se revisó además con capturas en celular, tablet y escritorio (ver ERS).

7. Cómo ejecutar las pruebas

```
npm install       # instala dependencias (Node.js 24)
npm test          # 94 pruebas + reporte de cobertura
# Resultado esperado: TOTAL: 94 SUCCESS
# Reporte: coverage/index.html
```

8. Conclusión

El proyecto cuenta con un proceso de testeo unitario completo: entorno configurado con Karma y Jasmine, pruebas para las funciones, hooks, componentes y flujos principales, uso de mocks para aislar dependencias (almacenamiento, diálogos, tiempo y navegación) y una cobertura medida de 100 % en sentencias, ramas, funciones y líneas, por sobre el mínimo de 90 % definido en la configuración.